Сравнение и анализ оптимизаций компиляторов

Шалахина Анна Олеговна

Группа: *25.Б43*
Кафедра: *системного программирования*
Научный руководитель: *Гориховский Вячеслав Игоревич*
Номер семестра практики: *2*

1. Введение

Компиляторы языка C оптимизируют один и тот же код по-разному, поэтому размер программы и набор инструкций получаются разными. В работе на двух тестовых программах сравниваются оптимизации компиляторов GCC, Clang и Intel ICX. Результатом являются таблицы сравнения ассемблерного кода.

2. Постановка задачи

Целью работы является сравнение и анализ оптимизаций, выполняемых компиляторами GCC, Clang и ICX при компиляции программ на языке C.

Задачи:

1. Выбрать сценарий, в котором может возникнуть необходимость применения языка C.
2. Выполнить обзор компиляторов, изучить выбранный алгоритм.
3. Определить характеристики системы, на которой будут проводиться исследования.
4. Выполнить компиляцию тестовых программ с оптимизациями по скорости и по размеру.
5. Произвести замеры объёма и времени выполнения исполняемых файлов.
6. Сгенерировать ассемблерные листинги для `-O0` и `-O2`.
7. Для каждого компилятора провести сравнение кода без оптимизации и с ней.
8. Сравнить стратегии оптимизации трёх компиляторов между собой.
9. Придумать несколько специфичных для сценария тестов и провести аналогичный эксперимент на них.

3. Обзор

Сценарий: проверка целостности данных в сетевых протоколах — вычисление контрольной суммы по массиву байт. Задача предполагает последовательную обработку большого массива данных и включает циклы, арифметические операции и обращение к элементам массива.

Тесты. Тест `optbench.c` разработан PC Tech Journal (1988) для проверки возможностей оптимизации компиляторов и охватывает 17 видов оптимизаций[5]. Для сценария выбран алгоритм контрольной суммы Флетчера[6, 7]. В сетевых протоколах он применяется, например, как альтернативная контрольная сумма TCP[8] и в транспортном протоколе ISO[9].

Компиляторы. Для сравнения выбраны GCC, Clang и Intel ICX.

- **GCC —** выбран как основной вариант, является самым распространённым компилятором с открытым кодом[1].
- **Clang —** компилятор языков семейства C, использующий инфраструктуру LLVM для оптимизации и генерации машинного кода[2].
- **Intel ICX —** компилятор C/C++ Intel oneAPI, также использующий LLVM[3]. Кроме того, для ICX установлена модель вещественной арифметики `-fp-model=fast`, позволяющая применять более агрессивные оптимизации операций с плавающей точкой[4]. Это различие следует учитывать при анализе соответствующих участков программы.

4. Описание предлагаемого решения

Система: EndeavourOS, ядро 7.2.6-arch2-1, архитектура x86_64, процессор AMD Ryzen 5 7430U with Radeon Graphics.
Инструменты: GCC 16.2.1, Clang 22.1.8, ICX 2026.0.0, `size` 2.47, `hyperfine` 1.20.0.

Каждая программа собирается с уровнями `-O0` (без оптимизаций), `-O2` (по скорости) и `-Os` (по размеру). `optbench.c` дополнительно собирается с флагом `-DNO_ZERO_DIVIDE`, исключающим блок деления на ноль. Ассемблерные листинги получены ключом `-S` для `-O0` и `-O2`.

По листингам составлены таблицы сравнения версий с оптимизацией и без для каждого компилятора, затем сводная таблица для `-O2`.

5. Эксперименты

5.1. Размер и время

Размер секций измерялся утилитой `size`, время утилитой `hyperfine` (среднее $\pm$ σ):

optbench.c

Размер секции `.text`, байт

Компилятор	-O0	-O2	-Os
GCC	3648	2949	2760
Clang	3330	3189	2836
ICX	3519	3733	2705

Время работы ($\pm\sigma$), мс

Компилятор	-O0	-O2	-Os
GCC	1,4±0,3	1,3±0,2	1,3±0,2
Clang	1,4±0,2	1,3±0,1	1,3±0,1
ICX	1,5±0,2	1,5±0,2	1,4±0,1

fletcher.c

Размер секции `.text`, байт

Компилятор	-O0	-O2	-Os
GCC	2761	2595	2174
Clang	2587	2965	2408
ICX	2325	3415	2091

Время работы ($\pm\sigma$), мс

Компилятор	-O0	-O2	-Os
GCC	1,6±0,4	1,4±0,2	1,5±0,2
Clang	1,6±0,3	1,4±0,2	1,5±0,3
ICX	1,7±0,2	1,5±0,1	1,6±0,1

5.2. Ассемблер, optbench.c

1. **Размножение констант и копий.** Во всех трёх компиляторах после `j4 = 2` в `printf` сразу передаётся число 2 (`movl $2, %esi`). Но после `printf` GCC и Clang снова загружают `j4` из памяти (`movl j4(%rip), %eax`). ICX сразу сравнивает `i2` с числом, учитывая, что `j4 = 2` (`cmpl $1, %r14d`). Значения `i2` и `i4` ICX держит в регистрах.
2. **Свёртка констант и арифметические тождества.** У всех трёх компиляторов результат одинаковый. `1 + 2` превратилось в 3 (`movl $3, i3`), сумма `2.4 + 6.3` записана числом. Тождества `i + 0`, `i / 1` и `i * 1` стали копированием `i` в `j2`, `k2` и `i4`, `i * 0` сразу ноль (`movl $0, i5`). Выражение `flt_6 * 0.0` ни один не заменил нулём, умножение осталось.
3. **Лишнее присваивание.** Во всех трёх листингах осталась одна запись (`movl $1, k3(%rip)`).
4. **Снижение мощности.** Умножение `4 * j5` во всех трёх компиляторах стало сдвигом влево на 2 бита. Цикл, который заполняет массив `ivector4` числами `i * 2`, исчез: массив `short` из 6 чисел (0, 2, 4, 6, 8, 10) записан двумя командами. Первая (`movq`/`movabsq`) кладёт сразу 4 числа (0, 2, 4, 6), вторая (`movl`) остальные 2 (8, 10).
5. **Простой цикл (do-while).** Все компиляторы вычислили результат цикла при компиляции. Осталось только `j5 = 10000` и `i5 = 0`.
6. **Управление переменной индукции цикла.** Переменная индукции: `i`. Для `ivector5[i * 2 + 3] = 5` компилятор не считает индекс заново на каждом шаге. GCC использует указатель, который сдвигается на 16 байт, и делает две записи за повтор. Clang использует счётчик, растущий на 4, и делает четыре записи за повтор. ICX использует смещение, которое растёт на 64 байта, и делает восемь записей за повтор, а конец цикла определяет по достижении нуля.
7. **Устранение ветвления и упрощение выражения.** Компиляторы записывают `i = 100` и оставляют только `k5 = i5 + i2`. Так как `i5` равно 0, остаётся `k5 = i2`: GCC и Clang читают `i2` из памяти, а ICX записывает готовое число (`movl $5, k5`).
8. **Генерация адреса, размножение копий и регистров.** Для постоянных индексов 0 и 2 адрес известен заранее (`ivector`, `ivector+8`). GCC и Clang один раз кладут адрес массива в регистр (%rbx и %r14), а `i2` читают заново для каждой записи. ICX сразу подставляет `i2 = 5` и пишет по готовому адресу `ivector+20`.
9. **Удаление общих подвыражений (CSE).** GCC и Clang считают сумму `h3 + k3` один раз и используют её и в условии, и в делении (для `g3` считают заново после `printf`). Два условия `< 0` и `> 5` они объединили в одну проверку (`cmpl $5` / `jbe` у GCC и `cmpl $6` / `jb` у Clang). ICX сразу считает `h3 + 1`. Деление на 3 заменено умножением на 2863311531 и сдвигом (`shrq $33`). У GCC и Clang деление осталось (`idivl`).
10. **Вынесение инвариантного кода (LICM).** ICX вычислил `j * k` один раз до цикла, а внутри остаётся только запись (`movb %bl`). GCC и Clang оставили умножение внутри цикла и каждый раз читают `j` и `k` из памяти. У GCC умножение 8-битное (`mulb`).
11. **Вызов функции с аргументами.** Во всех компиляторах в `main` вызова нет.
12. **Вызов функции без аргументов.** GCC оставил вызов `call unnecessary_loop`. Clang и ICX подставили тело функции прямо в `main`.
13. **`dead_code`: недостижимый код и лишние присваивания.** В функции переменная `idead_store` нигде не используется, а `if (0)` никогда не выполняется. На `-O2` у всех трёх компиляторов от функции остаётся только возврат (`ret` или `retq`).
14. **`unnecessary_loop`: удаление ненужного цикла.** Цикл повторяет пять раз одно и то же присваивание `k5 = x + j5` при `x = 0`. Во всех трёх компиляторах вместо него одна запись `k5 = j5` и `i = 5`. Вызов `printf` в конце превратился в переход `jmp printf`.
15. **`loop_jamming`: объединение циклов.** Объединить циклы нельзя без изменения порядка вывода: сначала печатаются пять значений первого цикла, потом пять второго. Поэтому GCC и Clang оставили два цикла (внутри умножения `imull`). ICX тоже не объединил циклы, но полностью развернул оба: вместо циклов идут подряд пять вызовов `printf`, потом ещё пять. Умножения на 2, 3 и 4 он заменил на `leal` и `shll $2`, а общее произведение `x * k5` посчитал один раз.
16. **`loop_unrolling`: разворачивание цикла.** GCC и Clang оставили цикл на 6 повторов. ICX развернул цикл полностью: в листинге шесть вызовов `printf` подряд, а нули в массив записаны тремя инструкциями (`movq $0` записывает четыре числа, и два `movw $0`).
17. **`jump_compression`: сжатие цепочки переходов.** На `-O0` хорошо заметны цепочки переходов (например, `jmp .LBB5_7`, затем `jmp .LBB5_9`, затем `jmp .LBB5_12`). На `-O2` все три компилятора ведут переходы сразу в нужное место и собирают общую сумму в одном месте в конце, а переменные держат в регистрах, а не в стеке. В `main` GCC оставил вызов функции, у Clang и ICX вызова нет.

Вывод. Все три компилятора выполняют базовые оптимизации: свёртку констант, применение арифметических тождеств, снижение мощности и удаление мёртвого кода. Различия проявляются в отдельных преобразованиях. В рассмотренном тесте ICX вынес умножение $j * k$ из цикла и заменил деление на константу умножением и сдвигом. В некоторых циклах `optbench.c` ICX выполнил полное разворачивание, тогда как GCC и Clang сохранили циклические конструкции. GCC также сохранил вызов `unnecessary_loop`, а Clang и ICX встроили тело этой функции в `main`. Ни один из компиляторов не объединил рассмотренные циклы.

5.3. Ассемблер, функции Флетчера

1. При `-O2` все три компилятора заменили операции вычисления остатка от деления на константу последовательностями арифметических инструкций, включающими умножение и сдвиги.
2. На `-O0` переменные `sum1`, `sum2`, `c0`, `c1`, `index`, `data` лежат в стеке, и каждая итерация читает и пишет их в память. При `-O2` компиляторы преимущественно используют регистры для хранения рабочих переменных, сокращая необходимость обращения к памяти.
3. Размотка циклов:

- GCC циклы не разматывает: одна итерация это один байт (для `fletcher16`) или одно число (для `fletcher32_opt`).
- Clang разматывает циклы: в `fletcher16` в 8 раз, в `fletcher16_opt` и `fletcher32_opt` в 6 раз.
- ICX разматывает циклы в 8 раз.

6. Заключение

В ходе работы получены следующие результаты:

1. Выбраны компиляторы GCC 16.2.1, Clang 22.1.8, Intel ICX 2026.0.0 и два теста: `optbench.c` и функции Флетчера, применяемые в сетевых протоколах.
2. Измерены размер секции `.text` и время работы для `-O0`, `-O2`, `-Os`. В обоих тестах у каждого из трёх компиляторов наименьший размер получен при `-Os`. Время выполнения программ при разных уровнях оптимизации различается незначительно; полученные значения с учётом стандартного отклонения близки.
3. Составлены таблицы сравнения ассемблерного кода для каждого компилятора и сводные таблицы для двух тестов.
4. Сопоставлены подходы компиляторов:
   - **GCC** выполняет базовые оптимизации, но сохранил вызов именно функции `unnecessary_loop` и не разворачивает циклы, деление заменяет умножением уже на `-O0`.
   - **Clang** выполняет базовые оптимизации, встраивает функции; в `fletcher.c` разворачивает цикл `fletcher16` в 8 раз, а циклы `fletcher16_opt` и `fletcher32_opt` — в 6 раз; на `optbench.c` циклы не разворачивает.
   - **Intel ICX** в рассмотренных тестах активнее разворачивает циклы, использует регистры для рабочих переменных и применяет дополнительные преобразования арифметических выражений. При этом размер секции `.text` при `-O2` оказывается больше, чем у GCC и Clang, для обоих тестов.

Репозиторий с исходным кодом, ассемблерными листингами и таблицами: `https://github.com/shannami/educational-practice`

Список литературы

1. Using the GNU Compiler Collection (GCC). — URL: `https://gcc.gnu.org/onlinedocs/` (дата обращения: 01.10.2026).
2. Clang Compiler User's Manual // Clang Documentation. — URL: `https://clang.llvm.org/docs/UsersManual.html` (дата обращения: 01.10.2026).
3. Invoke the Compiler // Intel oneAPI DPC++/C++ Compiler Developer Guide and Reference. — Версия 2026.0. — Intel Corporation. — URL: `https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/developer-guide-reference/2026-0/invoke-the-compiler.html` (дата обращения: 01.10.2026).
4. fp-model, fp // Intel oneAPI DPC++/C++ Compiler Developer Guide and Reference. — Версия 2026.0. — Intel Corporation. — URL: `https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/developer-guide-reference/2026-0/fp-model-fp.html` (дата обращения: 01.10.2026).
5. Хислей Ф. Н. Генерация высококачественного кода для программ, написанных на Си : перевод статьи из PC Tech Journal. — 1988.
6. Fletcher J. G. An Arithmetic Checksum for Serial Transmissions // IEEE Transactions on Communications. — 1982. — DOI: `https://doi.org/10.1109/TCOM.1982.1095369`.
7. Nakassis A. Fletcher's Error Detection Algorithm: How to Implement It Efficiently and How to Avoid the Most Common Pitfalls // ACM SIGCOMM Computer Communication Review. — 1988. — DOI: `https://doi.org/10.1145/53644.53648`.
8. Zweig J., Partridge C. TCP Alternate Checksum Options . — RFC 1146. — Internet Engineering Task Force, 1990. — URL: `https://www.rfc-editor.org/rfc/rfc1146.html` (дата обращения: 13.10.2026).
9. ISO. ISO Transport Protocol Specification, ISO DP 8073 . — RFC 905. — Internet Engineering Task Force, 1984. — URL: `https://www.rfc-editor.org/rfc/rfc905.html` (дата обращения: 13.10.2026).
10. Контрольная сумма Флетчера // — URL: `https://ru.wikipedia.org/wiki/Контрольная_сумма_Флетчера` (дата обращения: 14.10.2026).